VisionGuard

AI-generated and real image classification

MLOps Assignment 2

Submission	Details
Name	Syed Muhammad Hanzala
Roll number / project	23L-0907 / 23L-0907_A2_Ops
Report date	26 October 2026
Graded implementation	90b93195d46d58ecfcae174062cf6f5fef8c28bf
GitHub	https://github.com/Hanzala-Shadow/23L-0907_A2_Ops
DagsHub	https://dagshub.com/Hanzala-Shadow/23L-0907_A2_Ops
Live model API	https://23l-0907-a2-ops-api.vercel.app/predict
Public health check	https://23l-0907-a2-ops-api.vercel.app/health
Local web app	http://127.0.0.1:5000/

Project outcome

VisionGuard audits and versions the dataset, compares a custom CNN with MobileNetV2, tracks experiments on DagsHub, and serves the selected model through a cloud API. The local Flask interface sends uploads to that API and records results in PostgreSQL. The tuned model achieved 72.11% test accuracy and 0.8161 ROC-AUC.

Report scope

Tasks 1-8 are covered with the main decisions, results and essential evidence. Detailed commands, full screenshots and machine-readable receipts remain in the repository; docs/assignment-checklist.md maps every requirement to its evidence. The optional Grad-CAM bonus was not attempted.

Assistance disclosure

OpenAI Codex assisted with setup, implementation, debugging, experiments, deployment and report drafting. Results are backed by saved scripts, metrics and execution receipts.

Both repositories are private; grading access will be shared separately. The prediction API requires a key, which must be shared privately. The implementation revision above is unchanged by this shortened report.

Task 1 - Data and versioning

1.1 Environment

An isolated .venv uses Python 3.12.3, TensorFlow CPU 2.19.1, Keras 3.15.1, Keras Tuner 1.4.8, MLflow 2.22.5 and DVC 3.67.1. TensorFlow detected no GPU. Pinned requirements, bounded CPU threads and pre-operation disk checks keep execution reproducible within the 2 GiB free-space reserve (Figure 1).

1.2 Dataset audit and splits

Class	Files	Width min/med/max	Height min/med/max	Bytes min/med/max
ai_generated	539	225 / 1024 / 6000	135 / 800 / 4001	11738 / 199760 / 5626183
real	436	183 / 920.0 / 8495	158 / 720.0 / 7091	11311 / 144840.0 / 10195352

The custom walker records class, bytes, dimensions, colour mode, format and SHA-256. All 975 files decode: 0 corrupt files, 1 exact duplicate pair (image33.jpg/image34.jpg). Raw files are preserved; image34.jpg is excluded from the 974-image split inventory.

Split	AI-generated	Real-labeled	Total
Train	376	305	681
Validation	81	65	146
Test	81	66	147

Seed-42 hash ranking produces stratified 70/15/15 lists. Validation checks confirm no path or hash overlap and identical lists on reproduction. Exact hashing does not detect near-duplicates.

1.4 DVC storage and toy restore

The toy CSV has two committed versions on the dvc-lab branch. After checking out the older version and deleting both its local data and cache, dvc pull restored the original bytes (Figure 2). Its pointer holds the checksum, size and path; large content stays outside Git. Project data, splits and model bundles use DagsHub DVC storage; credentials remain ignored.



Figure 1. Recorded versions and GPU check.



Figure 2. Two toy commits and verified old-version restore.

Task 1 - Visual exploration

1.3 Samples and distributions

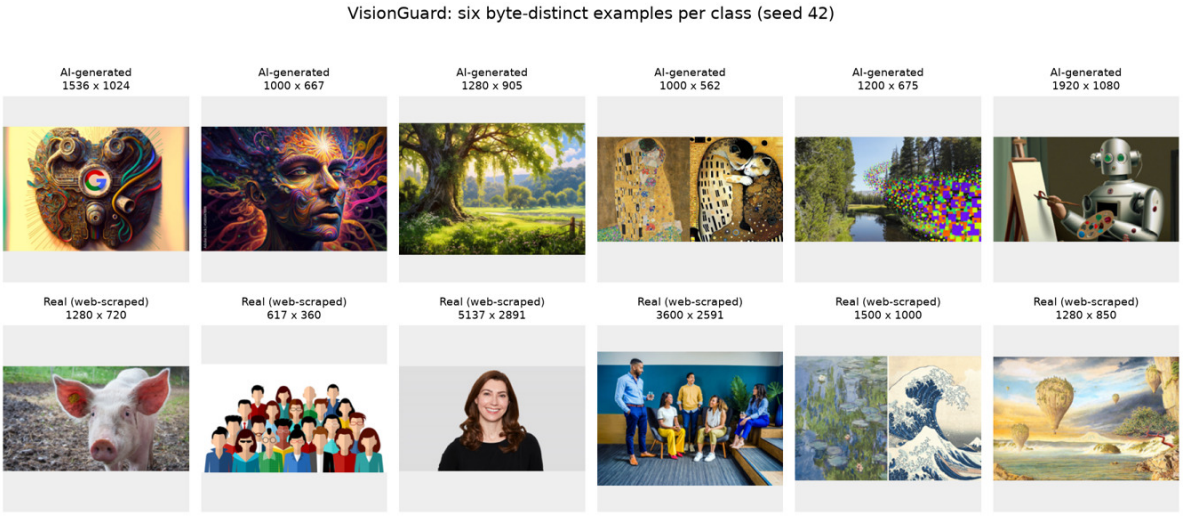


Figure 3. Seeded examples from both folder labels.

The grid includes photographs, artwork, composites and web graphics. Folder labels define the task target, but image origin was not independently verified; this diversity can confuse the model.

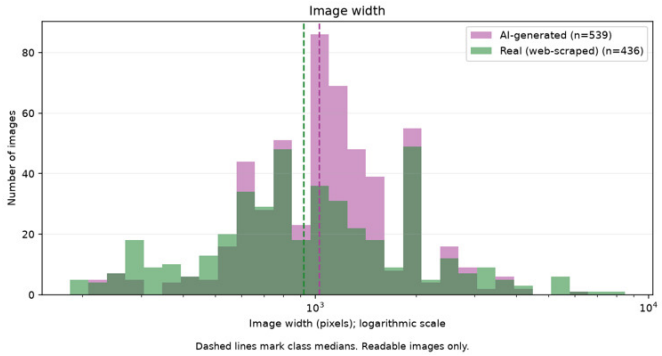


Figure 4. Image widths by class.

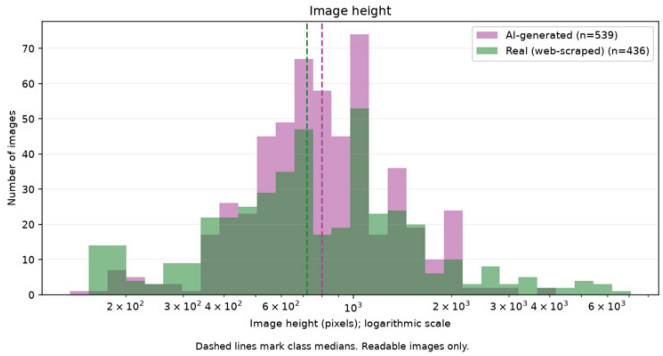


Figure 5. Image heights by class.

Widths: Both classes span small images to several thousand pixels, with common generation sizes visible in the AI class. Resizing bounds CPU work. Heights: Portrait and landscape shapes vary substantially. Letterboxing preserves aspect ratio instead of stretching subjects.

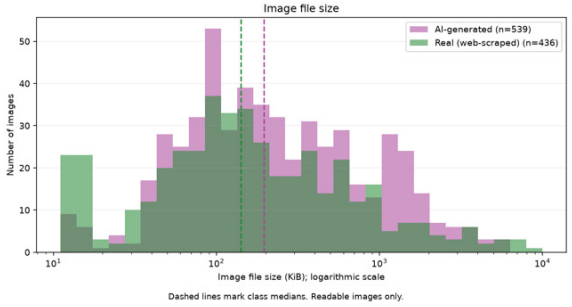


Figure 6. Per-class file sizes, showing a long tail.

File sizes depend on compression and content as well as dimensions. Byte limits alone cannot bound decoded memory, so serving also checks pixel count.

Task 2 - Pipeline and CNN

2.1 Input pipeline

Split CSVs feed tf.data. The shared decoder corrects EXIF orientation, converts to RGB, composites transparency on white and letterboxes to 128 x 128 with bilinear resizing. This preserves aspect ratio and limits CPU cost. Decoded uint8 tensors are cached in RAM (about 39 MiB), training is shuffled with seed 42, then batched by 16 and prefetched one batch.

Training alone uses horizontal flips, rotation up to 18 degrees and zoom up to 10%. Validation and inference use the unaugmented decoder; normalization is saved inside the model.

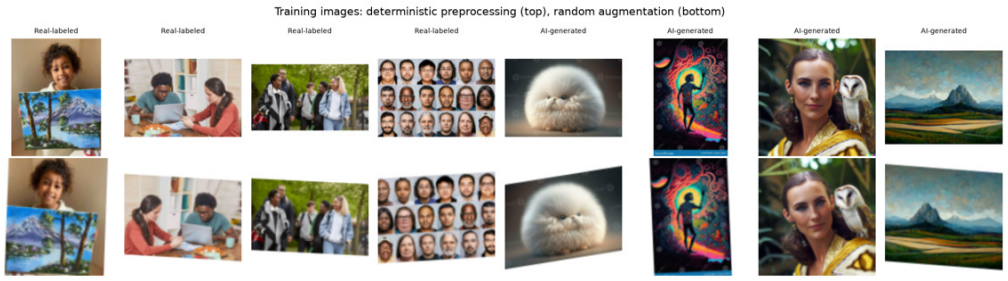


Figure 7. Original and augmented training examples.

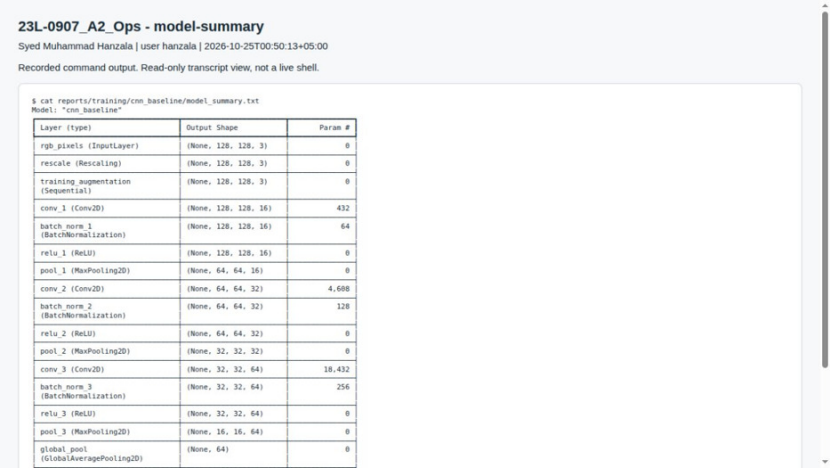
2.2 Custom CNN architecture

Layer	Type	Output shape (batch omitted)	Parameters
rgb_pixels	InputLayer	(128, 128, 3)	0
rescale	Rescaling	(128, 128, 3)	0
training_augmentation	Sequential	(128, 128, 3)	0
conv_1	Conv2D	(128, 128, 16)	432
batch_norm_1	BatchNormalization	(128, 128, 16)	64
relu_1	ReLU	(128, 128, 16)	0
pool_1	MaxPooling2D	(64, 64, 16)	0
conv_2	Conv2D	(64, 64, 32)	4608
batch_norm_2	BatchNormalization	(64, 64, 32)	128
relu_2	ReLU	(64, 64, 32)	0
pool_2	MaxPooling2D	(32, 32, 32)	0
conv_3	Conv2D	(32, 32, 64)	18432
batch_norm_3	BatchNormalization	(32, 32, 64)	256
relu_3	ReLU	(32, 32, 64)	0
pool_3	MaxPooling2D	(16, 16, 64)	0
global_pool	GlobalAveragePooling2D	(64)	0
head	Dense	(64)	4160
dropout	Dropout	(64)	0
ai_probability	Dense	(1)	65

Convolutions learn local features; BatchNorm stabilizes activations; ReLU adds nonlinearity; pooling reduces spatial cost. Global averaging limits parameters, Dense(64) combines features, dropout 0.3 regularizes, and a sigmoid gives AI probability. Total parameters: 28,145.

Task 2 - Transfer learning

The CNN uses pixel/255 normalization, binary cross-entropy and Adam at 0.001. Accuracy, precision, recall and ROC-AUC expose different error types. Its recorded model summary confirms the layer table on the preceding page (Figure 8).



23L-0907_A2_Ops - model-summary

Syed Muhammad Hanzala | user hanzala | 2026-10-25T00:50:13+05:00

Recorded command output. Read-only transcript view, not a live shell.

```
$ cat reports/training/cnn_baseline/model_summary.txt
Model: "cnn_baseline"
```

Layer (type)	Output Shape	Param #
rgb_pixels (InputLayer)	(None, 128, 128, 3)	0
rescale (Rescaling)	(None, 128, 128, 3)	0
training_augmentation (Sequential)	(None, 128, 128, 3)	0
conv_1 (Conv2D)	(None, 128, 128, 16)	432
batch_norm_1 (BatchNormalization)	(None, 128, 128, 16)	64
relu_1 (ReLU)	(None, 128, 128, 16)	0
pool_1 (MaxPooling2D)	(None, 64, 64, 16)	0
conv_2 (Conv2D)	(None, 64, 64, 32)	4,608
batch_norm_2 (BatchNormalization)	(None, 64, 64, 32)	128
relu_2 (ReLU)	(None, 64, 64, 32)	0
pool_2 (MaxPooling2D)	(None, 32, 32, 32)	0
conv_3 (Conv2D)	(None, 32, 32, 64)	18,432
batch_norm_3 (BatchNormalization)	(None, 32, 32, 64)	256
relu_3 (ReLU)	(None, 32, 32, 64)	0
pool_3 (MaxPooling2D)	(None, 16, 16, 64)	0
global_pool (GlobalAveragePooling2D)	(None, 64)	0

Figure 8. Recorded CNN summary excerpt; the full output is saved in model_summary.txt.

2.3 Frozen backbone and last-block fine-tuning

ImageNet MobileNetV2 is loaded without its classifier, followed by global averaging, Dense(64), dropout 0.3 and a sigmoid. Its saved normalization is pixel/127.5 - 1, matching the pretrained backbone. Applying CNN-style normalization would change its input distribution.

Stage	Trainable layers / parameters	Adam rate
Frozen	New dense classifier only; 82,049 trainable parameters	0.001
Fine-tuned	Head + block_16_expand, block_16_depthwise, block_16_project; 551,489 parameters	0.00001

Every BatchNorm layer, earlier block and Conv_1 remain frozen. The second stage starts from the best frozen checkpoint and recompiles the optimizer. Inference-mode backbone statistics avoid changing BatchNorm estimates; saved weight checks confirm the intended boundaries.

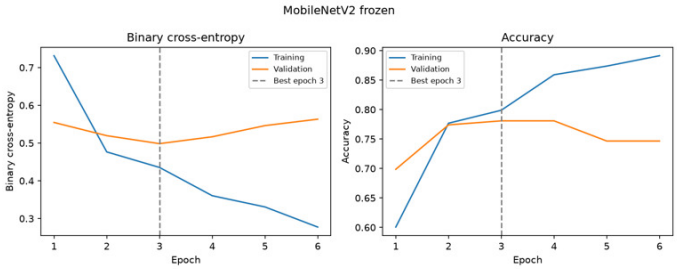


Figure 9. Frozen-head curves; best validation epoch marked.

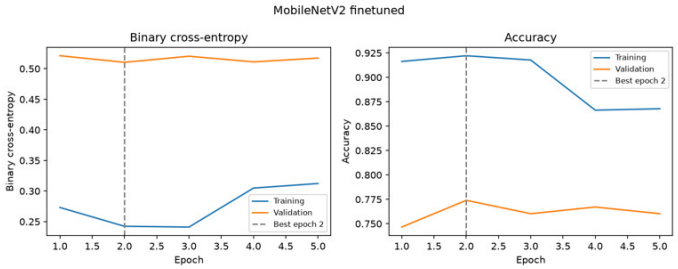


Figure 10. Last-block fine-tuning did not improve validation loss.

Task 2 - Training and tuning

2.4 Training discipline

Seed 42 fixes Python, NumPy and TensorFlow randomness. Deterministic operations, bounded threads, early stopping, ReduceLROnPlateau and checkpoints on minimum validation loss are used. Best epochs: CNN 1/6, frozen MobileNet 3/6, fine-tuned MobileNet 2/5. Only saved best checkpoints are compared; exact cross-hardware equivalence is not assumed.

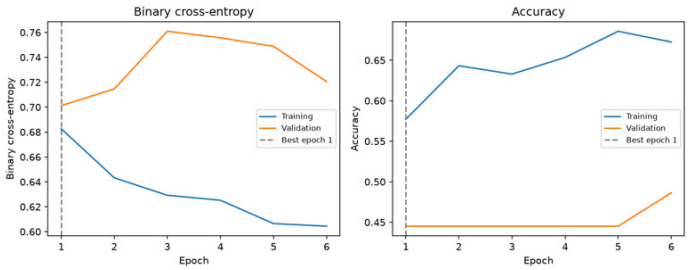


Figure 11. CNN loss and accuracy with best epoch 1 marked.

2.7 Keras Tuner: toy lab and eight project trials

The MNIST toy uses 6,000 training and 1,000 validation examples from its training partition, with RandomSearch over learning rate and hidden units. Four three-epoch trials are logged to DagsHub.

Toy trial	Learning rate	Units	Val. loss	Val. accuracy
0	0.01	32	0.3145	0.9000
1	0.001	32	0.4416	0.8840
2	0.01	64	0.2470	0.9150
3	0.001	64	0.3765	0.8960

Trial	Learning rate	Dropout	Unfrozen convs	Val. loss	Best epoch
0	0.0001	0.2	1	0.51384	4
1	1e-05	0.5	1	0.50332	2
2	1e-05	0.5	3	0.50416	2
3	0.0001	0.2	3	0.54409	4
4	1e-05	0.2	3	0.51258	2
5	1e-05	0.2	1	0.51104	2
6	0.0001	0.5	1	0.49604	4
7	0.0001	0.5	3	0.48514	3

All eight project trials start from the same frozen checkpoint. Trial 7 wins: rate 0.0001, dropout 0.5 and three unfrozen convolutions, validation loss 0.48514. Higher dropout has the largest average effect (loss lower by 0.02322). Repeated selection can overfit validation; the test set was never used for tuning.

Task 2 - Final evaluation

2.5 Model selection and threshold

Minimum validation loss selects trial 7 from eleven candidates. Threshold 0.6196606457 minimizes 2 x false positives + false negatives on validation: calling a real image fake is assigned twice the cost of missing an AI image. This is a product assumption. The CNN threshold is 0.4725604951. Both choices were locked before the single test evaluation per model.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
cnn_baseline	0.4014	0.2308	0.0370	0.0638	0.4669
champion	0.7211	0.7857	0.6790	0.7285	0.8161

Test contains 147 images (81 AI, 66 real). The champion has TN=51, FP=15, FN=26, TP=55; the CNN has TN=56, FP=10, FN=78, TP=3. Figures 12-15 use saved test probabilities; hash-bound receipts prevent another inference pass.

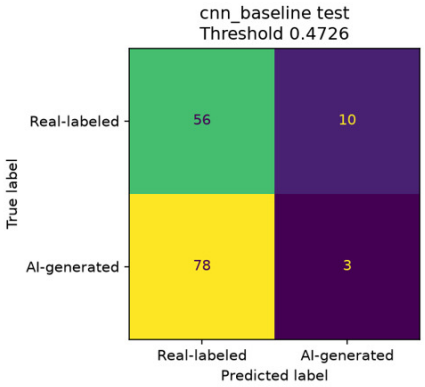


Figure 12. CNN confusion matrix.

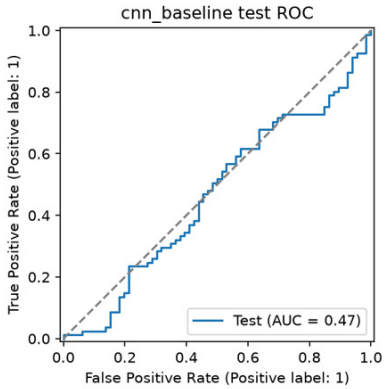


Figure 13. CNN ROC curve.

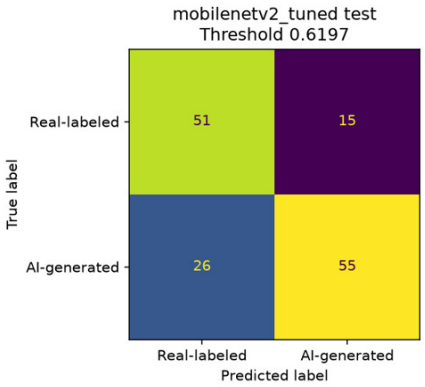


Figure 14. Champion confusion matrix.

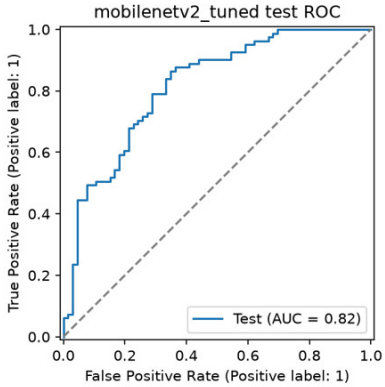


Figure 15. Champion ROC curve.

Champion accuracy fell from 80.82% on validation to 72.11% on test. Small, diverse data and repeated validation selection limit generalization. The weak CNN result is retained, and test performance did not trigger further tuning. Predictions do not prove image origin.

Task 2 - Error analysis

2.6 Ten misclassified test images

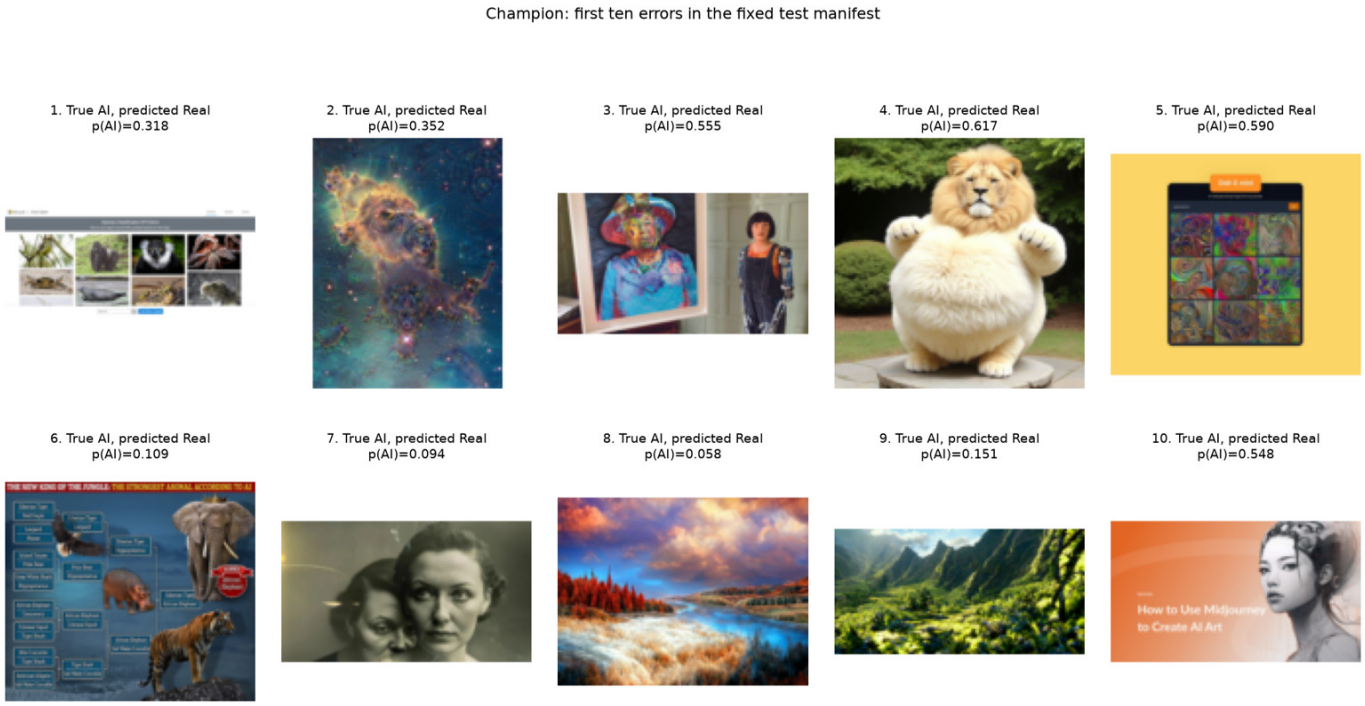


Figure 16. First ten errors in fixed test-manifest order, selected from the saved evaluation.

All ten are false negatives because AI-labeled paths appear first in the manifest. They are not representative of every error type; the full test also has 15 false positives.

Example	Visual hypothesis
1	Multi-photo web layout may favor familiar photographic cues.
2	Nebula-like painterly texture overlaps with real-labeled artwork.
3	Photographic framing may dominate the small generated artwork.
4	Fur and garden textures look familiar; 0.617 lies just below the cutoff.
5	Small artworks in a product graphic lose detail after resizing.
6	Infographic text and cutouts differ from an isolated generated image.
7	Grainy monochrome portrait may resemble a real photograph.
8	Saturated landscape shares scene structure with real-labeled artwork.
9	Plausible mountain scenery may outweigh synthesis artifacts.
10	Banner text and small portrait dominate; probability 0.548 misses the cutoff.

These are visual hypotheses, not causal explanations. They suggest future data review without changing the locked evaluation.

Task 3 - MLflow and DagsHub

3.1 Local tool lab; 3.2 DagsHub setup

A 16-unit MNIST network trains on 1,200 examples with 200 validation examples for two epochs. Learning rates 0.001 and 0.01 give validation accuracy 0.54 and 0.80, respectively. Both local runs log parameters, epoch metrics, model and loss plot. Repeating the 0.001 run on DagsHub reproduces 0.54 accuracy (Figures 17-18).

An experiment groups attempts; a run records one attempt; parameters describe configuration; metrics measure performance; artifacts are saved files; tags record context. Private DagsHub mirrors GitHub and provides the project MLflow tracking URI. Credentials are read from environment variables in ignored .env files.

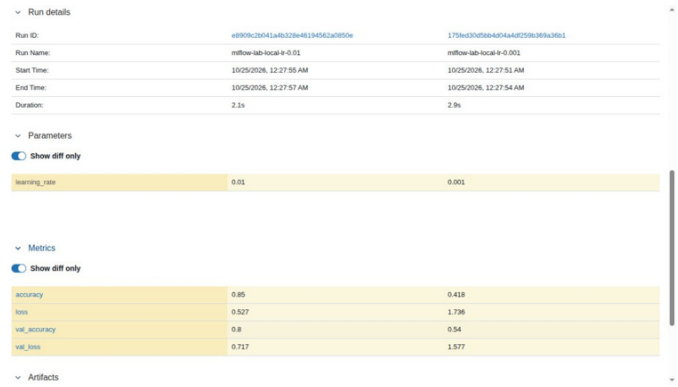


Figure 17. Local MLflow toy comparison.

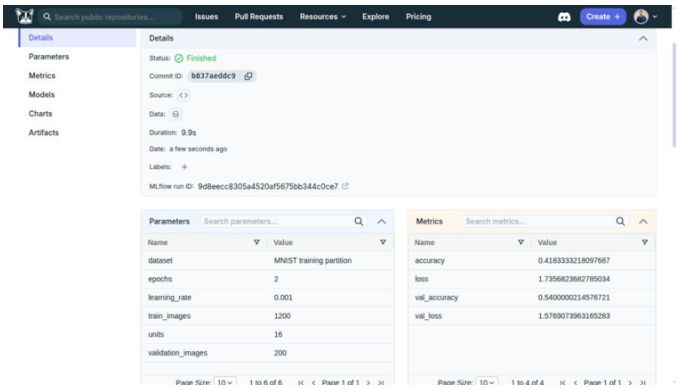


Figure 18. Verified DagsHub toy run.

3.3 Manual logging and TensorFlow autologging

The CNN callback manually logs epoch loss, accuracy, precision, recall, AUC and learning rate. MobileNet uses TensorFlow autologging for fit configuration and epoch metrics. Both explicitly add seed, Git revision, DVC hash, split lists and the best checkpoint. Auto-upload of the final model is disabled so it cannot replace the best checkpoint; tuner trials log explicitly.

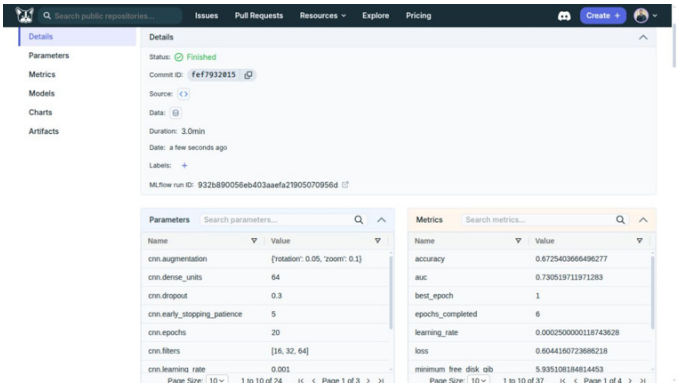


Figure 19. CNN manual logging.

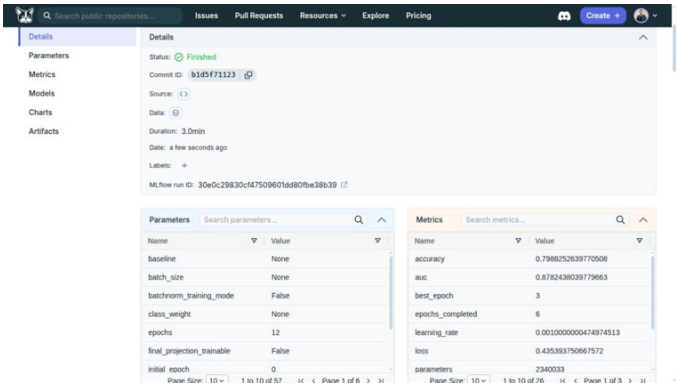


Figure 20. Frozen MobileNet autologging.

A local/remote tracking-address mix-up in the toy lab was corrected by capturing the cloud URI before switching to local storage. Only the independently verified remote run is claimed here.

Task 3 - Comparison and registry

3.4 Artifacts; 3.5 Meaningful comparison

Runs retain model, configuration, packages, split lists, layer summary, training curves and validation outputs. Final evaluation stores both models' test plots and saved predictions, with final metrics attached to the originating training runs. The table compares eleven candidates; six distinct tuning runs are also shown in Figure 22.

Candidate	Val. loss	Accuracy at 0.5	Val. ROC-AUC
cnn_baseline	0.70121	0.4452	0.5151
mobilenetv2_frozen	0.49845	0.7808	0.8418
mobilenetv2_finetuned	0.51045	0.7740	0.8427
tuner_0	0.51384	0.7671	0.8439
tuner_1	0.50332	0.7671	0.8431
tuner_2	0.50416	0.7671	0.8429
tuner_3	0.54409	0.7603	0.8496
tuner_4	0.51258	0.7466	0.8397
tuner_5	0.51104	0.7534	0.8389
tuner_6	0.49604	0.7603	0.8498
tuner_7	0.48514	0.7877	0.8663

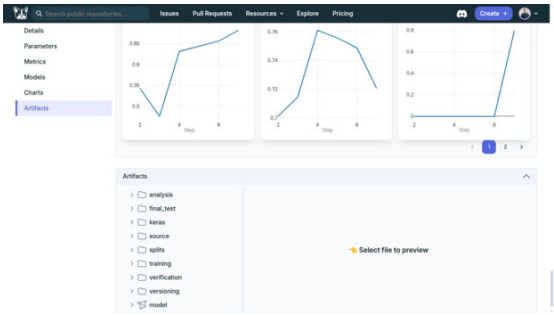


Figure 21. Saved model and reporting artifacts.

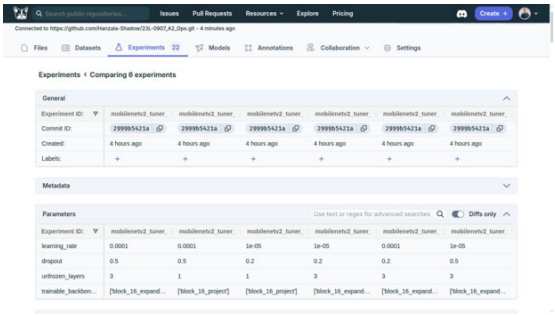


Figure 22. Six runs with distinct tuning parameters.

3.6 Registry and independent alias load

Registry 23L-0907_A2_Ops contains version 1 (frozen MobileNet) and version 2 (tuned MobileNet); alias champion points to version 2. The separate script scripts/predict_registry.py loads models:/23L-0907_A2_Ops@champion and predicts a validation image: probability 0.9492709637 at saved threshold 0.6196606457 (Figures 23-24).

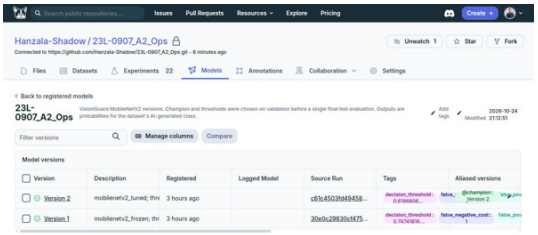


Figure 23. Two versions and champion alias.

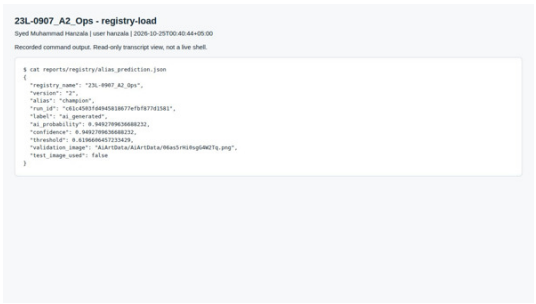


Figure 24. Recorded independent alias-load prediction.

Tasks 4-5 - Flask and PostgreSQL

4.1 Model loading; 4.2 Routes and guards

Native serving loads and validates the champion once per worker and uses the training decoder. The retained native image and model remain available. Current Flask uses one remote client per worker; the cloud interpreter loads once per warm instance. Two original validation-image checks and all 146 validation export-parity decisions are recorded; no test data was reused.

Routes provide upload, prediction, saved result, paginated history and health. Upload checks cover form token, one file, sanitized name, type, decoding, 4 MiB bytes and 4 million pixels. JPEG, PNG and WebP stay in memory; only metadata and outcomes are stored.

4.3 Interface; 4.4 Configuration

HTML/CSS presents upload, verdict/confidence and history (Figures 25-28). Environment variables supply database, model policy, server secrets and cloud settings. Debug is off; the development Flask server was checked, and Compose runs Gunicorn with one worker and two threads. A configured threshold must match the saved policy; the API key stays server-side.

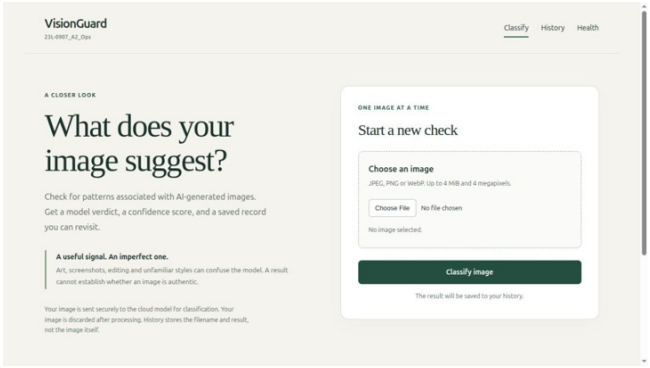


Figure 25. Local upload page.

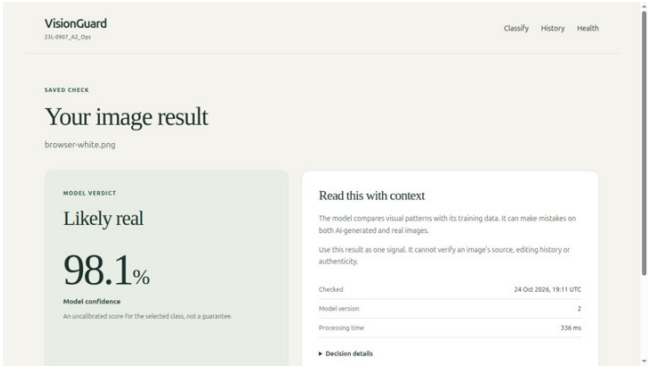


Figure 26. Saved verdict, confidence and version.

Task 5 - Persistent database trail

PostgreSQL records UTC time, safe filename, image hash/size, status, label, probability, confidence, threshold, latency and model version/identity. Rejections and cloud failures have safe error fields and no verdict. SQLAlchemy transactions back the history menu; an external named volume preserves rows across recreation. All 11 original rows survived; eight integration checks brought the total to 19.

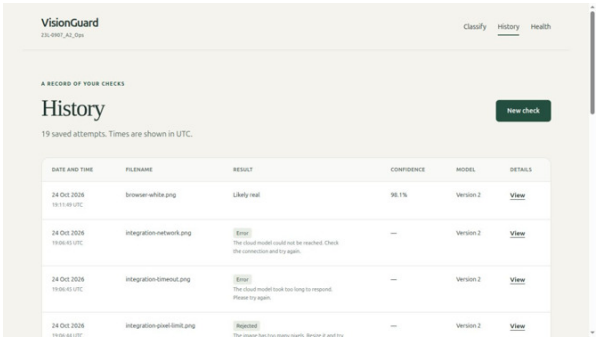


Figure 27. PostgreSQL-backed history.

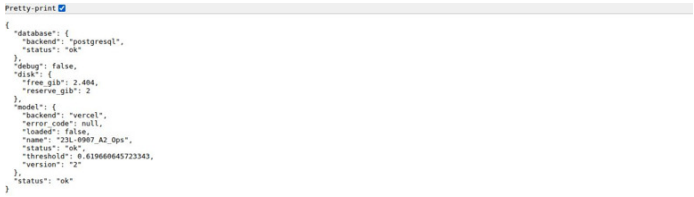


Figure 28. Healthy database, remote model and debug off.

Task 6 - Docker Compose

6.1 Ready-made image lab

An isolated postgres:17-alpine lab exercised run, ports, volume, logs, shell, listing, stop, start, restart and cleanup. An image is a template; a container is its instance; a layer is a cached build change; a volume outlives a container; port mapping forwards a host port. Only lab resources were removed. This warm-up and the DVC toy were completed during final audit.

6.2 Image design; 6.3 Compose configuration

Component	Configuration and reason
Images	Pinned Python slim; dependencies copied before source for caching; UID 10001; CPU-only native image and smaller remote client.
Build context	Allowlisted serving files; exclude secrets, dataset, weights, Git metadata and training tools; no pip cache.
Services	web + db; web uses db:5432 and waits for database health. Health probes check readiness.
Persistence	Existing external PostgreSQL volume; read-only champion/policy mount; read-only root with 128 MiB tmpfs.
Ports / secrets	Web 127.0.0.1:5000 and DB admin 127.0.0.1:5433; ignored environment file; no public database port.

6.4 Size reduction and layers

docker image ls reports native 1.72 GB versus client 395 MB, about 77% smaller. Moving inference out of the web image removes TensorFlow, trading offline use for a network dependency. docker history shows the largest install layer falling from 1.22 GB to 63.9 MB.

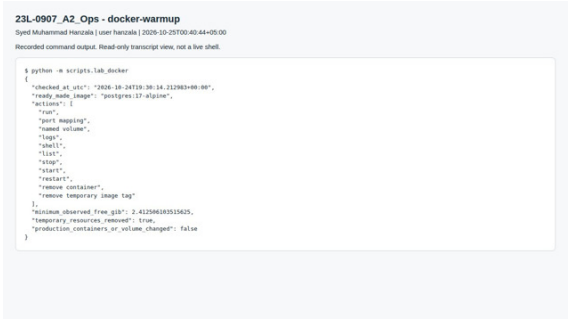


Figure 29. Recorded Docker warm-up and cleanup.



Figure 30. Healthy services and preserved mounts.

6.5 Failure diagnosis and recovery

In the isolated native lab, a missing model mount caused FileNotFoundError for policy.json. Adding the read-only bundle restored HTTP 200, loaded=true and healthy PostgreSQL (Figure 31). Logs, shell and lifecycle commands were checked; earlier database recreation preserved history.



Figure 31. Actual missing-policy log and corrected health response.

Task 6 - Automated checks

6.6 Failing-then-passing CI lab

The private 23L-0907_A2_Ops-ci-lab repository runs Python 3.11 and 3.12. The first commit deliberately expects 2+2=5 and fails; correcting the expectation to 4 makes both versions pass (Figures 32-33). Each result belongs to a separate committed workflow run.

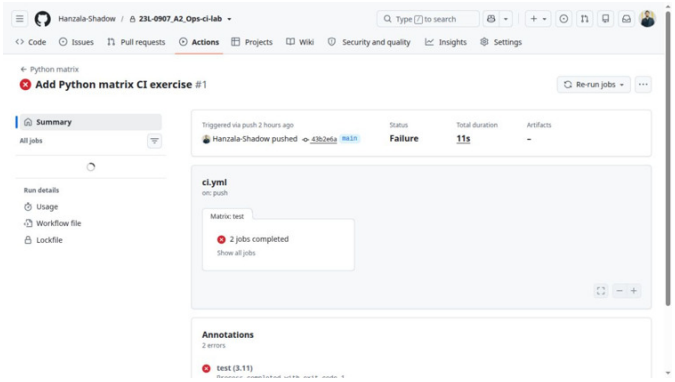


Figure 32. Expected failure on both Python versions.

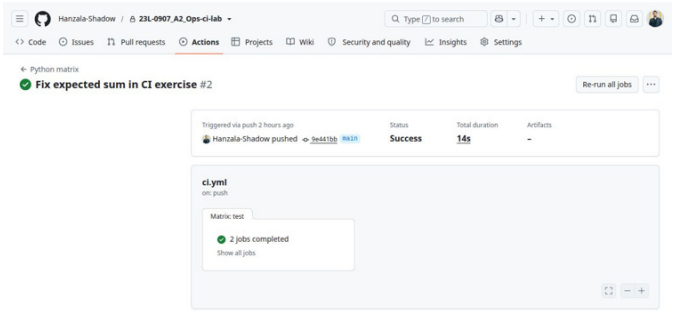


Figure 33. Corrected test passes on both versions.

Project tests and Docker build checks

Main CI checks repository hygiene, Ruff, synthetic tests, native/client Docker builds and an isolated Compose upload/persistence test. The graded implementation passed 141 tests and Docker/Compose checks. The README includes its CI badge; the screenshot below shows the project workflow, and the exact graded-revision run is linked below.

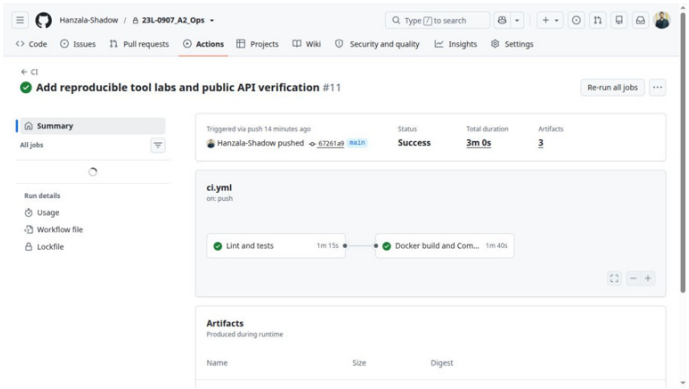


Figure 34. Passing main project workflow with build and test jobs.

Graded-revision CI run 40127571425. Routine CI uses generated runner credentials. The separately authorized cloud API key is an encrypted GitHub Actions secret used only by the manual Postman check; it is never embedded in source or reports.

Reproduction and handoff

Check free space with scripts/check_disk.py, restore missing assets using dvc pull, then use docker compose up -d --no-build --wait with the configured cloud override. Full setup is in README.md and docs/cloud-integration.md. The report builder reads saved results only. Do not rerun the one-time test evaluator to regenerate the report.

For submission: review this PDF, grant the grader access to both private repositories and share the API key privately if required. Source pushes contain no credentials, dataset/model binaries, agent files or decorative icons.

Task 7 - Cloud deployment

Vercel Hobby hosts the key-protected model API while Flask and PostgreSQL stay local. The float32 LiteRT export avoids shipping TensorFlow in the function and matched every validation decision. Render required card verification; Vercel was used without paid add-ons.

Step	Deployment action
1	Lock champion and threshold; export float32 LiteRT and verify validation parity.
2	DVC-version the export; prepare a model-only archive and pin its SHA-256 manifest.
3	Create the Hobby Python project in fra1 with pinned lightweight dependencies.
4	Set cloud.wsgi:app; fetch and verify the model bundle during build.
5	Supply a short-lived model-only URL and a new API key as encrypted settings.
6	Deploy reviewed source; confirm READY and public health. POST /predict requires the key.
7	Run the Postman collection from a separate GitHub-hosted network; retain safe results.



Figure 35. Public cloud readiness with model identity.

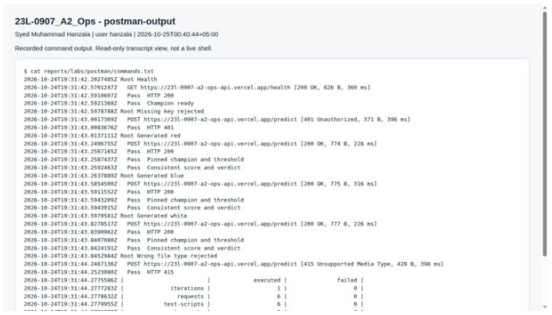


Figure 36. Actual external Postman CLI result.

External Postman validation

Request	HTTP	Response ms
Health	200	360
Missing key rejected	401	396
Generated red	200	228
Generated blue	200	316
Generated white	200	226
Wrong file type rejected	415	398

Postman CLI 1.76.0 ran six requests and 13 assertions with zero failures on a hosted Ubuntu runner. Generated red, blue and white images test wiring, not accuracy. The versioned collection is postman/visionguard.postman_collection.json.

Live URL: https://23l-0907-a2-ops-api.vercel.app/health. Cold starts and free-tier quotas affect availability; a new build needs a refreshed model-download URL. Automatic Git deployment is disabled.

Task 8 - Local-to-cloud integration

After local validation, Flask sends the image over verified HTTPS to Vercel. The client uses a 3-second connect timeout and 45-second read timeout, no redirects and no automatic POST retries. It bounds JSON to 16 KiB and checks version, hashes, saved threshold, score, verdict, confidence and dimensions before recording success.

Generated input	AI probability	Verdict	History
Red	0.0782903	real	saved success
Blue	0.1078499	real	saved success
White	0.0190690	real	saved success

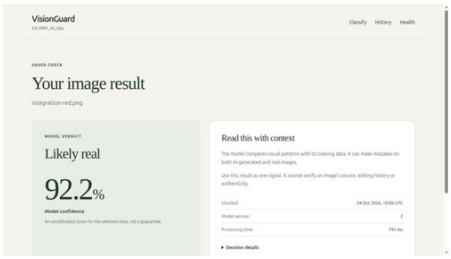


Figure 37. Red input result.

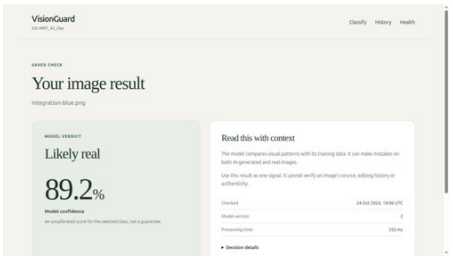


Figure 38. Blue input result.

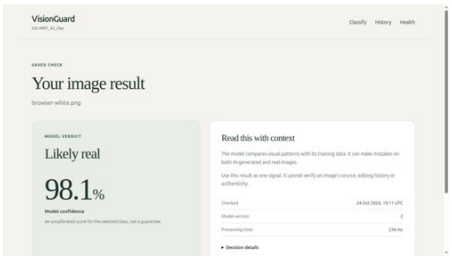


Figure 39. White input result.

All three generated inputs matched the local LiteRT reference within 0.0001. The browser upload/result/history flow was checked; no dataset image was sent for these integration tests.

Condition	HTTP	Outcome
Wrong / corrupt / oversized file	415 / 400 / 413	Friendly saved rejection
Connect or read timeout	504	cloud_timeout; no verdict
Network, authentication or busy service	503	Safe error and retry guidance
Invalid or mismatched response	502	Rejected score; no success row

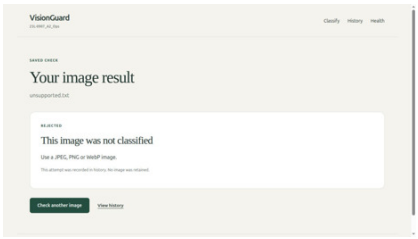


Figure 40. Wrong type rejected.

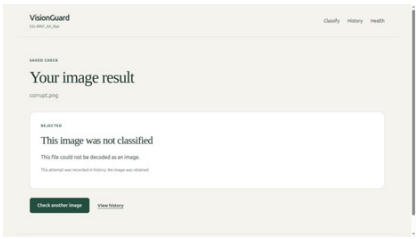


Figure 41. Corrupt image rejected.

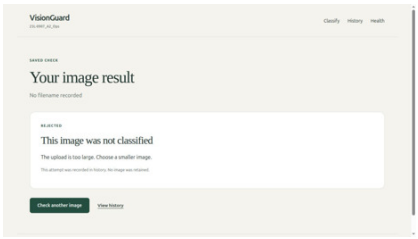


Figure 42. Oversized image rejected.

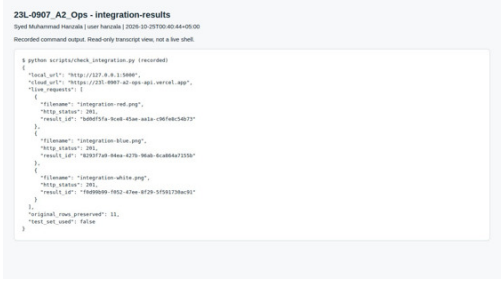


Figure 43. Recorded cloud results and database IDs.

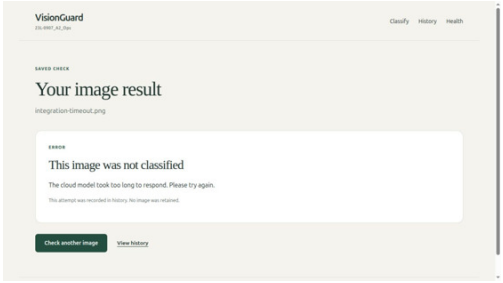


Figure 44. Saved timeout with a readable history link.

Injected timeout/network failures wrote error rows without interrupting production services. Original PostgreSQL rows and native model files remained unchanged. Uploads are not saved, logs rotate and disk guards preserve the 2 GiB reserve.

References: Kaggle dataset; Keras; MLflow; DVC; Docker Compose; GitHub Actions; Vercel; Postman CLI. Full evidence mapping: docs/assignment-checklist.md.